Assignment - 1

Assigned: February 27, 2026 Due: March 13, 2026 Max. marks: 100

General Instructions

- Submit a hard-copy of the solutions after the class or in TAs office NAC - 2, Room 311.
- Upload a scanned copy of the solutions and the codes you have developed as a single zipped folder on Moodle.
- Do not email your solutions/codes. Emailed submissions will not be graded.

Hand Calculations

Programming Questions

1. (30 points) Consider writing a program to evaluate the following integral using Reimann integration and Monte Carlo methods:

$$I = \int_{x=0}^{4}\int_{y=0}^{3}\int_{z=0}^{2}(4x^3 + xy^2 + 5y + yz + 6z)\,dz\,dy\,dx. \tag{1}$$

In Reimann integration, the value of an interal can be numerically approximated as Reimann sums as follows,

$$\iiint_V f(x,y,z)\,dz\,dy\,dx = \sum_{i,j,k} f(x^*_{ijk}, y^*_{ijk}, z^*_{ijk})\Delta v^*_{ijk}, \tag{2}$$

where $x^*_{ijk}, y^*_{ijk}, z^*_{ijk}$ are the sample points within the integration volume, V, and v^*_{ijk} is the incremental volume of the sampling space used. While Reimann integration can be extended to solve higher dimesional integrals, it quickly becomes computationally intensive and alternate method of integration such as Monte Carlo methods that are based on probability are often used. In Monte Carlo method, the integral can be represented as follows,

$$\iiint_V f(x,y,z)\,dz\,dy\,dx = V\overline{f} \pm V f_{\text{rms}} \tag{3}$$

where $\overline{f}$ is the average value of f over the entire region, and f_{rms} is the standard deviation or room mean square value of the function and they can be calculated as follows,

$$\overline{f} = \frac{\sum_{l=1}^{N} f(x_l, y_l, z_l)}{N}, \tag{4}$$

$$f_{\text{rms}} = \frac{\sqrt{\overline{f^2} - \left(\overline{f}\right)^2}}{N}, \tag{5}$$

$$\overline{f^2} = \frac{\sum_{l=1}^{N} f(x_l, y_l, z_l)^2}{N}. \tag{6}$$

$$\tag{7}$$

This approximation will hold good as the number of sampling points used in calculating the average value tends to infinity. In the above equations N represents the total number of random points (x_1, y_1, z_1), (x_2, y_2, z_2), ..., (x_N, y_N, z_N). The accuracy of the final result depends on N as well as the quality of the random number generator. For every point, l, the random number generator needs to be called three times to get random values for x_l, y_l and z_l that lie within resptive domain extents.

(a) Develop a serial program to calculate the integral using Reimann integration. Test your program for $N_x = 48$, $N_y = 36$ and $N_x = 24$. Compare the value obtained with the analytical result. Parallelize this code using OpenMP and test for $p = 2, 4$ and 8 threads. Compare the computational time of the serial program with each of the parallel runs. Ensure that the serial and the parallel programs produce correct and the same output.

(b) Develop a serial program to calculate the integral using Monte Carlo algorithm. Run your program using $N = 100, 1000, 10^4, 10^5$ and 10^6 points. Parallelize this code using OpenMP and re-run for the same values of N using $p = 2, 4$ and 8. Compare the execution times of the serial and the parallel programs.

2. (70 points) Consider the solution of the following Poisson equation,

$$\nabla^2\phi = -q; \quad q = 2(2 - x^2 - y^2); \quad \phi(\pm 1, y) = 0; \quad \phi(x, \pm 1) = 0; \tag{8}$$

using Jacobi method. The discretized equation can be written as follows,

$$\phi_{i,j}^{(k+1)} = \frac{1}{4}\left[\phi_{i+1,j}^{(k)} + \phi_{i-1,j}^{(k)} + \phi_{i,j+1}^{(k)} + \phi_{i,j-1}^{(k)}\right] + \frac{\Delta^2}{4} q_{i,j} \tag{9}$$

where $\Delta = \Delta x = \Delta y$. Consider an initial guess of $\phi(x, y) = 0$ everywhere. The exact solution to this problem is given by $\phi = (x^2 - 1)(y^2 - 1)$. Use double precision arithmetic.

(a) Develop a serial Jacobi program to solve the discretized equations and solve the system using $\Delta = \Delta x = \Delta y = 0.1$ (that is 21 points along each of the directions). Report the number of iterations required to bring the numerical solution to within 1% of the exact solution. Plot the numerical and the analytical solutions of ϕ vs x for $y = 0.5$.

(b) Develop an OpenMP program for Jacobi method.

(c) For the parallel program developed above, verify that the parallel program is giving the same result as that of the serial program that you developed above for a grid of $\Delta = 0.1$ by plotting the two results on the same graph. Upon successful verification, run this program for $\Delta = 0.1, 0.01$, and 0.005 using 8 threads. Plot the time-taken by the parallel solvers and the serial solver as a function of the grid size (Δ). Do you see any improvement in performance?

(d) For a grid size of $\Delta = 0.005$, plot the time-taken by each of the parallel solvers as a function of the number of threads. Consider the number of threads to be $p = 2, 4, 8$ and 16.